Data Science & AI

DATA STRUCTURES Through Python

Queues

Lecture No.- 03

By- Satya sir

Recap of Previous Lecture

- Circular Queue

$$f = (f+1) \% N$$

$$r = (r+1) \% N$$

Full : $f == (r+1) \% N$

Empty : $f == -1$

- Double Ended QUEue (DEQUE)

	rear	front
Enqueue	$r = (r+1) \% N$	$f = f-1$
Dequeue	$r = r-1$	$f = (f+1) \% N$

Topics to be Covered

- Deque operations Example
- Priority Queue
- Queue Implementation

Topic : Double Ended QUEue (DEQUE)

Let Deque DQ[7], f=-1, r=-1, N=7

0	1	2	3	4	5	6
~~10~~	~~40~~	~~60~~	~~70~~	~~50~~	~~30~~	~~20~~

~~r=2~~ ~~f=3~~

Enqueue_rear(10) r=0, f=0

Enqueue_front(20) r=0, f=6

Enqueue_front(30) r=0, f=5

Enqueue_rear(40) r=1, f=5

Enqueue_front(50) r=1, f=4

Enqueue_rear(60) r=2, f=4

Enqueue_front(70) r=2, f=3

Enqueue_rear(80) ⇒ f==(r+1)%N True overflow

Dequeue_rear() ⇒ Delete 60, r=1, f=3

Dequeue_front() ⇒ Delete 70, r=1, f=4

Dequeue_rear() ⇒ Delete 40, r=0, f=4

Dequeue_rear() ⇒ Delete 10, r=6, f=4

Dequeue_rear() ⇒ Delete 20, r=5, f=4

Dequeue_front() ⇒ Delete 50 r=5, f=5

Dequeue_front() ⇒ Delete 30 if f==r f=r=-1

Dequeue_rear()
(or)
Dequeue_front()
f==-1
Underflow

Topic : Priority Queue

⇒ It is used in Binary Heap Implementation

It is a special form of Queue, in which Deletion is based on Priority of Element.

	0	1	2	3	4	r=5
Item	~~A~~	~~B~~	C	~~D~~	~~E~~	~~F~~
Priority	3	5	1	2	4	6

r=5, f=5

1 = Low Priority

6 = high Priority

N=6 PQ = { }

$\boxed{r=(r+1)\%N}$

PQ.append('A' : 3) r=0, f=0

PQ.append('B' : 5) r=1, f=1

PQ.append('C' : 1) r=2, f=1

PQ.append('D' : 2) r=3, f=1

PQ.append('E' : 4) r=4, f=1

PQ.append('F' : 6) r=5, f=5

Insertion Sequence

A→B→C→D→E→F

Deletion Sequence

F→B→E→A→D→C

Dequeue() : Delete 'F', r=4 f=1

Dequeue() : Delete 'B', r=4, f=4

Dequeue() : Delete 'E', r=3 f=0

Dequeue() : Delete 'A', r=3 f=3

Dequeue() : Delete 'D', r=2 f=2

Dequeue() : Delete 'C', r=-1, f=-1

Topic : Queue Implementation

Drawback: Contiguous memory allocation, memory space may get wasted.

implementing Queue using List :

```
q=[]
q.append(10)
q.append(100)
q.append(1000)
q.append(10000)
print("Initial Queue is:",q) #[10 100 1000 10000]
print(q.pop(0)) 10
print(q.pop(0)) 100
print(q.pop(0)) 1000
print("After Removing elements:",q) [10000]
```

```
def is_empty():
    return len(q) == 0

def size():
    return len(q)

print("Check queue is empty or not", is_empty())
print("Size of the queue:", size())
```

FALSE

1

Topic : Queue Implementation

Drawback : Not fit for multithreading applications

```
# implement queue using queue module

from queue import Queue  class
q=Queue(maxsize=4)
print("Initial Size Before Insertion:",q.qsize())# 0
q.put('A')
q.put('AA')
q.put('AAA')
q.put('AAAA')
print("After Insertion:",q.qsize())  # 4
print("Queue is Full or Not:",q.full()) # True
print("Size of Queue:",q.qsize()) # 4
```

```
print("Removing Elements:")
print(q.get())  # A
print(q.get()) # AA
print(q.get()) # AAA
print("Empty or Not??",q.empty()) # FALSE
print(q.get()) # AAAA
print("Empty or Not??",q.empty()) # TRUE
print("Size of Queue:",q.qsize()) # 0
```

~~A~~	~~AA~~	~~AAA~~	~~AAAA~~

Topic : Queue Implementation

Implementation Using deque ~~module~~ class

```
from collections import deque
q=deque()
q.append(10)
q.append(100)
q.append(1000)
q.append(10000)
print("Initial Queue is:",q)
print(q.popleft())
print(q.popleft())
print("After Removing elements:",q)
```

module

class

10	100	1000	10000

f r

10

100

[1000, 10000]

Topic : Queue Implementation - 2

Simple Queue

```
class Queue:
    def __init__(self, size): #Constructor
        self.size = size
        self.queue = [None] * size
        self.front = self.rear = -1
```

```
def is_empty(self):
        return self.front == -1
```

```
def is_full(self):
        return (self.rear == self.size + 1)
```

Topic : Queue Implementation - 2

Simple Queue

```
def enqueue(self, item):
    if self.is_full():
        print("Queue is full")
        return
    elif self.is_empty():
        self.front = self.rear = 0
    else:
        self.rear = (self.rear + 1)
    self.queue[self.rear] = item
```

```
def dequeue(self):
    if self.is_empty():
        print("Queue is empty")
        return
    removed_item = self.queue[self.front]
    if self.front == self.rear:
        self.front = self.rear = -1
    else:
        self.front = (self.front+1)
    return removed_item
```

f=0 f=1 f=2 f=3 r=3

~~10~~	~~20~~	~~30~~	~~40~~

Topic : Queue Implementation - 2

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = self.rear = -1
```

```
def is_empty(self):
        return self.front == -1
```

```
def is_full(self):
        return (self.rear + 1) % self.size == self.front
```

Topic : Queue Implementation - 2

```
def enqueue(self, item):
    if self.is_full():
        print("Queue is full")
        return
    elif self.is_empty():
        self.front = self.rear = 0
    else:
        self.rear = (self.rear + 1) % self.size
    self.queue[self.rear] = item
```

```
def dequeue(self):
    if self.is_empty():
        print("Queue is empty")
        return
    removed_item = self.queue[self.front]
    if self.front == self.rear:
        self.front = self.rear = -1
    else:
        self.front = (self.front + 1) % self.size
    return removed_item
```

Topic : Queue Implementation - 2

```
class Deque:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def addFront(self, item):
        self.items.append(item)
```

From last

```
    def addRear(self, item):
        self.items.insert(0,item)
```

From First

```
    def removeFront(self):
        return self.items.pop()
```

From last

```
    def removeRear(self):
        return self.items.pop(0)
```

From first

```
    def size(self):
        return len(self.items)
```

front

Enqueue

```
d=Deque()
print(d.isEmpty()) # True
d.addRear(4)
d.addRear('dog')
d.addFront('cat')
d.addFront(True)
print(d.size()) # 4
print(d.isEmpty()) # FALSE
d.addRear(8.4)
print(d.removeRear())
print(d.removeFront())
```

Topic : Queue Implementation - 2

Priority Queue

1 = High
4 = Low

```
Employees=[]
Employees.append((2,"A"))
Employees.append((3,"B"))
Employees.sort(reverse=True)
Employees.append((1,"C"))
Employees.sort(reverse=True)
Employees.append((4,"D"))
Employees.sort(reverse=True)
```

```
While employees:
    print(employees.pop(0))
```

While list is Not Empty

C, A, B, D

Priority	2	3	1
Item	A	B	C

Sort() ⇒

High — Low Priority

1	2	3	4
C	A	B	D

append(1, 'E') ⇒

2	3	4	1
A	B	D	E

Sort()

1	2	3	4
E	A	B	D

Delete() ⇒ 'E'

Topic : Queue Implementation - 2

```
from queue import PriorityQueue
p = PriorityQueue()
```

Class

object Priority

1 = high , 6 = low

```
p.put((2, "ABC"))
p.put((1, "IJK"))
p.put((4, "PQR"))
p.put((6, "XYZ"))
```

2	1	4	6	3
ABC	IJK	PQR	XYZ	DEF

```
next = p.get() # return Item with high Priority by default
print(next) # IJK

print(p.empty()) # FALSE
print(p.full()) # FALSE
```

```
p.put((3, "DEF"))
print(p.get()) # ABC

while not p.empty():
    print(p.get()) # DEF
                   PQR
                   XYZ

print(p.empty())
```

TRUE

Topic : Queue Implementation - 2

Priority Queue

```
import heapq (module)

X= []

heapq.heappush(X, (3, 'ABC'))

heapq.heappush(X, (1, 'IJK'))

heapq.heappush(X, (2, 'PQR'))

while X:
        item = heapq.heappop(X)
        print(item)
```

1 IJK
2 PQR
3 ABC

2 mins Summary

- Deque operations
- Simple queue
  Circular queue
  Deque
  Priority queue

  } Implementation

THANK - YOU